SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering

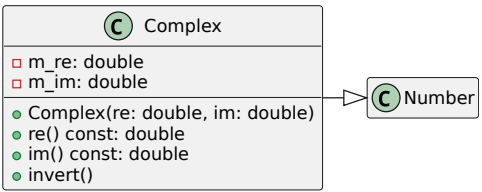


11. C++ Klassen, Operatoren & Vererbung

11.1 C++ Klassen

Klassen & Sichtbarkeiten (i)

- Klassen in C++
  - haben Datenkomponenten
  - haben Methoden (Funktionen auf eigene Daten)
  - keine, eine oder mehrere Basisklassen → Mehrfachvererbung




```
class Number {};

class Complex : public Number {
  private:
    double m_re {};
    double m_im {};
  public:
    Complex(double re = 0, double im = 0);
    double re() const;
    double im() const;
    void invert();
};
```

- Elemente von Klassen haben Sichtbarkeiten (`private`, `protected`, `public`)
  - werden für Gruppe von Elementen (*member*) vergeben (nicht einzeln wie in Java)

Klassen & Sichtbarkeiten (ii)

- es gibt zwei (fast) gleichwertige Schlüsselwörter: `class` und `struct`
  - einziger Unterschied ist die Sichtbarkeit
    - `class` → default `private`
    - `struct` → default `public`
- folgende Klassendeklarationen somit identisch
  - da Sichtbarkeiten explizit

```
class Complex {
  private:
    double m_re { };
    double m_im { };
  public:
    Complex(double re = 0, double im = 0);
    void invert();
};
```

```
struct Complex {
  private:
    double m_re { };
    double m_im { };
  public:
    Complex(double re = 0, double im = 0);
    void invert();
};
```

Methoden

- Funktion mit Objekt als erstem Parameter
- wenn möglich als `const` Methode (erlaubt Aufruf auf konstantes Objekt)
- Definition entweder
  - innerhalb der Klassendefinition
  - oder außerhalb mit Klassennamen als Präfix

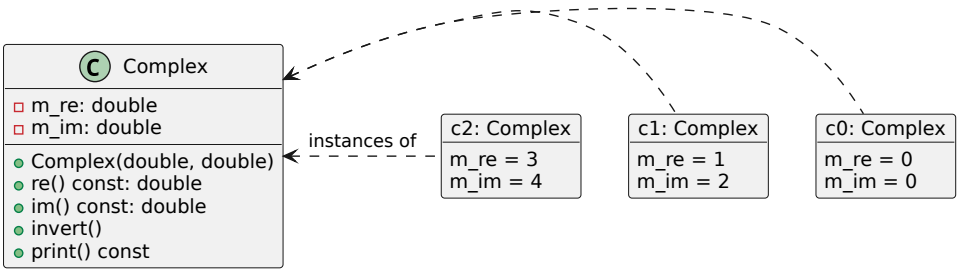
```
class Complex {
  void print() const; // declaration
  void invert() { /* inline definition */ }
};
void Complex::print() const {
  // out-of-class definition
}
```

- Objekt selbst ist impliziter erster Parameter `*this`
- Mentales Modell als C-Funktion

```
void Complex::print(const Complex * const this); // const pointer to const Complex
void Complex::invert(     Complex * const this); // const pointer to Complex
```

Objekte

- Objekte sind Instanzen von Klassen
- Struktur (ähnlich zu C-`struct`) wird von Klasse vorgegeben
- Werte unterscheiden sich von Objekt zu Objekt
  - jedes Objekt braucht Speicher für Datenkomponenten
- Methoden sind Funktionen mit Objekt als implizitem Parameter
  - verbrauchen nur einmalig Speicher (im Codesegment)
  - Objekte haben keine eigene Kopie von Methoden



Friends

- befreundete Klassen, Methoden oder Funktionen haben Zugriff auf private Elemente

```cpp
class Secret {

  friend void show_secret(Secret s);
  friend class Revealer;

  private:
    int value {123};
};

struct Revealer {
  static void show_secret(Secret s) {
    SHOW(s.value); // access private value
  }
};
```

```cpp
void show_secret(Secret s) {
  SHOW(s.value); // access private value
}

void no_access(Secret s) {
  // SHOW(s.value); XXX: compile error
}

int main() {
  Secret s {};
  show_secret(s);
  Revealer::show_secret(s);
  no_access(s);
}
```

```
s.value = 123
s.value = 123
```

Konstruktor und Destruktor

```
class Complex {
  private:
    float m_re, m_im; // not initialized
  public:

    Complex() {
      PL("Complex constructor");
    }

    ~Complex() {
      PL("Complex destructor");
    }

    float re() const { return m_re; }
    float im() const { return m_im; }
};

Complex c; // NOTE: implicit constructor call
SHOW(c.re());
SHOW(c.im());
// NOTE: implicit destructor call at end of scope
```

- spezielle Methoden für Initialisierung und Freigabe
  - Konstruktor `Complex()` für Initialisierung
  - Destruktor `~Complex()` für Freigabe
- vom Compiler automatisch generiert
  - falls nicht implementiert
- werden automatisch aufgerufen

```
Complex constructor
c.re() = -1.6947395e+38
c.im() = -1.6947395e+38
Complex destructor
```

Konstruktor-Notation

- expliziter Konstruktoraufruf per Konvention mit geschwungenen Klammern {}
  - vermeidet Verwechslung mit Funktionsdeklaration, z.B. `Number create();`
- auch direkte Zuweisung bei Initialisierung ist Konstruktoraufruf

```
struct Number {
  double m_n;
  Number(double n = 0) { m_n = n; }
};

Number n1_a {1}; // canonical constructor call
Number n1_b = 1; // NOTE: also constructor call
SHOW(n1_a.m_n);

// NOTE: also valid but unusual
Number n1_c = {1};        // XXX: avoid
Number n1_d = Number {1}; // XXX: avoid
Number n1_e = Number(1);  // XXX: avoid
```

```
n1_a.m_n = 1
```

https://compiler-explorer.com/z/7MzaqK5Ke

Standardkonstruktor (*default constructor*)

```
class Complex {
  private:
    double m_re {1}; // NOTE: direct init
    double m_im {2}; // no constructor necessary
  public:
    double re() const { return m_re; }
    double im() const { return m_im; }
};

Complex c; // automatic constructor call
Complex c2 {}; // explicit constructor call
SHOW(c.re(), c.im());
SHOW(c2.re(), c2.im());
```

```
c.re() = 1; c.im() = 2
c2.re() = 1; c2.im() = 2
```

- Konstruktor ohne Parameter für alle Klassen
  - automatisch generiert
    - falls möglich und kein eigener Konstruktor definiert
  - automatisch aufgerufen
- Datenkomponenten möglichst direkt initialisieren
  - keine speziellen Methoden notwendig
  - “Rule-of-Zero”
- kein automatischer Struct-Initializer wie in C

```
Complex c = {1, 2}; // XXX: no struct-initializer
Complex c {1, 2}; // XXX: no initializer list
```

The rule of three/five/zero - cppreference.com

Initialisierungsliste (i)

- Compiler garantiert, dass alle Datenkomponenten im Konstruktorrumpf bereits initialisiert sind
  - Initialisierungsliste ermöglicht Werte davor bereits einzusetzen
  - verhindert doppelte Initialisierung

```
class Complex {
  private:
    double m_re;
    double m_im;
  public:
    Complex(double re = 0, double im = 0)
      : m_re(re), m_im(im) {
      std::println("Complex({}, {})", re, im);
    }
    double re() const { return m_re; }
    double im() const { return m_im; }
    void re(double re) { m_re = re; }
    void im(double re) { m_re = re; }
};
```

Initialisierungsliste (ii)

- ohne Initialisierungsliste
  - 1. Default-Initializer ausführen
  - 2. Werte zuweisen

```
class Calc1 {
    Complex m_value;
  public:
    Calc1(double start = 0)
      // NOTE: compiler initializes m_value
      //        before entering constructor
    {
      std::println("Calc1({})", start);
      // NOTE: m_value is assigned a new value
      m_value = Complex(start, start);
    }
};
Calc1 c1{1};
```

- mit Initialisierungsliste
  - sofort Werte zuweisen

```
class Calc2 {
  private:
    Complex m_value;
  public:
    Calc2(double start = 0)
      : m_value(start, start) {
      std::println("Calc2({})", start);
    }
};

Calc2 c2{2};
```

Impliziter Konstruktoraufruf

- jeder Konstruktor mit einem Parameter kann implizit aufgerufen werden[1]
  - z.B. um Argument einer Funktion zu erzeugen

```
struct Complex {
  int m_re, m_im;
  Complex(int re = 0, int im = 0)
    : m_re(re), m_im(im) {
    std::println("Complex({}, {})", re, im);
  }
};

void print(Complex c) {  // <---------------.
  SHOW(c.m_re, c.m_im);  //                 |
}                        //                 |
                         //             print(Complex(123))
int main() {  //                           |
  print(123); // implicit constructor call -'
}
```

```
Complex(123, 0)
c.m_re = 123; c.m_im = 0
```

- `explicit` markieren falls nicht gewünscht

```
struct Complex {
  int m_re, m_im;
  explicit Complex(int re = 0, int im = 0)
    : m_re(re), m_im(im) {}
};
```

- kein implizierter Aufruf mehr möglich

```
error: In function ‘int main()’:
could not convert ‘123’ from ‘int’ to ‘Complex’
   12 |  print(123);
      |        ^~~
      |        |
      |        int
```

[1] besser wäre umgekehrt mit leider nicht existierendem Schlüsselwort `implict`

Kopierkonstruktor & Zuweisungsoperator

```
1  class Complex {
2      double m_re;
3      double m_im;
4    public:
5      Complex(double re = 0.0, double im = 0.0)
6        : m_re(re), m_im(im) {}
7
8      // copy constructor
9      Complex(const Complex &c)
10       : m_re(c.m_re), m_im(c.m_im) {
11       PL("Complex({}, {}) copy", m_re, m_im);
12     }
13
14     // assignment operator
15     Complex& operator = (const Complex &c) {
16       PL("({}, {}) <- ({}, {})",
17          m_re, m_im, c.m_re, c.m_im);
18       m_re = c.m_re;
19       m_im = c.m_im;
20       return *this;
21     }
22 };
```

- spezieller Konstruktor zum Erzeugen einer Kopie: Kopierkonstruktor
- Zuweisung eines Werts an ein Objekt: Zuweisungsoperator

```
23 int main() {
24   Complex c1 {1, 2};
25   Complex c2 {c1}; // copy-constructor (cctor)
26   Complex c3 = c2; // cctor
27
28   // only normal ctor (copy-elision)
29   Complex c4 { Complex { 3, 4 } };
30   Complex c5 = Complex { 3, 4 };
31
32   c1 = c4; // assignment operator
33 }
```

```
Complex(1, 2) copy
Complex(1, 2) copy
(1, 2) <- (3, 4)
```

learncpp.com Class initialization and copy elision

Überblick automatische Klassen-Elemente (i)

- Compiler erzeugt mehrere Methoden (falls nicht vorhanden)
  - kann mit `= delete` unterdrückt werden
  - bzw. mit `= default` explizit gemacht werden

- Default-Konstruktor
  - Konstruktor ohne Parameter
  - ruft Default-Konstruktoren aller Datenkomponenten auf
  - z.B. `auto c = Complex()`
- Destruktor
  - ruft Desktrukoren aller Datenkomponenten auf
- Kopier-Konstruktor
  - erzeugt neues Objekt als Kopie eines anderen
  - z.B.
    - `Complex c2 = c1;`
    - `Complex c2{c1};`
    - `Complex c2(c1);`
- Zuweisungsoperator
  - verändert Objekt um es gleich zu machen
  - z.B. `c2 = c1;`

```
struct Complex {
    // default construtor
    Complex() = default;

    // destructor
    ~Complex() = default;

    // copy constructor
    Complex(const Complex&) = default;

    // assignment operator
    Complex& operator = (const Complex&) = default;
};
```

- Rückgabe von `*this` durch `operator = ()` um Verkettung zu ermöglichen
  - z.B. `c3 = c2 = c1;`

Überblick automatische Klassen-Elemente (ii)

Für Move-Semantik (effizientere Speicher-Wiederverwendung) werden außerdem noch automatisch folgende Methoden generiert (Details zu Move-Semantik im nächsten Kapitel)

- Move-Konstruktor
  - verschiebt Inhalt eines Objekts in ein neues
- Move-Zuweisungsoperator
  - verschiebt Inhalt eines Objekts in ein anderes
- Ausgangsobjekt wird dabei jeweils *“leer geräumt”*

```
struct Complex {

    // move constructor
    Complex(Complex&&) = default;

    // move assignment operator
    Complex& operator = (Complex&&) = default;
};
```

Automatische Ressourcenverwaltung

Ressourcenverwaltung durch Kapselung und automatische Variablen

```
struct Array {
  int *a;

  Array(std::size_t n)
    : a(new int[n]()) {
    std::println("allocated {} elements", n);
  }

  ~Array() {
    delete[] a;
    std::println("deleted all elements");
  }

  // NOTE: don't generate copy and assignment ops
  Array(const Array &)             = delete;
  Array& operator = (const Array &) = delete;
};

Array a(10);
// NOTE: automatic deallocation at end of scope
```

- Konstruktor reserviert Speicher
- Destruktor gibt frei
- Variable am Stack
  - automatische Freigabe
  - Desktruktoraufruf garantiert

```
allocated 10 elements
deleted all elements
```

Zuweisung & Kopieren jetzt problematisch

kopiert nur Adresse hinter Zeiger und nicht Speicher und Werte

a) entweder verbieten (`= delete`)

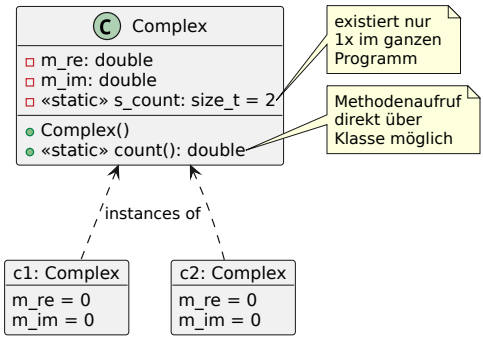
b) oder eigene Implementierung (siehe Move-Semantik im nächsten Kapitel)

Statische Klassenkomponenten (*static members*)

- statische Daten und Methoden an Klasse (und nicht an Objekte) gebunden

```
class Complex {
  double m_re, m_im;
  static size_t s_count;
public:
  Complex() {
    s_count++;
    std::println("Complex {} created", s_count);
  }
  static size_t count() { return s_count; }
};
size_t Complex::s_count {};
int main() {
  Complex c1, c2;
  SHOW(Complex::count());
}
```

```
Complex 1 created
Complex 2 created
Complex::count() = 2
```



11.2 C++ Operatoren überladen

Überladen von Operatoren

- C++ erlaubt Überladen von Operatoren
  - z.B. Rechenoperationen für eigenen Datentyp
  - Definition mit Schlüsselwort `operator`

```
class Complex { /* ... */ };

Complex operator + (Complex a, Complex b) {
  return Complex(a.re() + b.re(), a.im() + b.im());
}

int main() {
  Complex a {1, 2};
  Complex b {3, 4};
  Complex c = a + b; // --- syntactic sugar --.
  SHOW(c.re(), c.im()); //                    |
  c = operator+(a, b); // <------------ for --'
  SHOW(c.re(), c.im());
}
```

```
c.re() = 4; c.im() = 6
c.re() = 4; c.im() = 6
```

Überladen von Operator: Funktion vs. Methode

- manche Operatoren nur als Methode möglich
  - 1. Operand → Objekt selbst
  - 2. Operand → 1. Parameter

```
class Complex {
    // ...
    Complex& operator += (const Complex &o) {
      m_re += o.m_re;
      m_im += o.m_im;
      return *this;
    }
};
int main() {
  Complex a {1, 2};
  Complex b {3, 4};
  b += a; // --> syntactic sugar -.
  SHOW(b.re(), b.im()); //   for  |
  b.operator += (a); // <---------'
  SHOW(b.re(), b.im());
}
```

```
b.re() = 4; b.im() = 6
b.re() = 5; b.im() = 8
```

Liste von überladbaren Operatoren

- als Funktion oder Methode
  - Arithmetik `+`, `-`, `*`, `/`, `%`, `^`, `&`, `|`, `~`, `!`
  - Logik `&&`, `||` (ohne Kurzschlussevaluierung)
  - Vergleiche `<`, `>`, `==`, `!=`, `<=`, `>=`
    - seit C++20 *Spaceship*-Operator `<=>`
- nur als Methode
  - Zuweisungen `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `^=`, `&=`, `|=`, `>>=`, `<<=`
  - Dereferenzierung `->`, `->*` (Objekt als Pointer)
  - Funktionsaufruf `()` (Objekt als Funktion)
  - Index `[]` (Objekt als Array)
  - Inkrement `++` und Dekrement `--`
  - Sequenz `,`
- Priorität kann nicht verändert werden

Beispiel Index-Operator

- Objekt der Klasse kann wie normales Array verwendet werden

```
class Complex {
  private:
    double m_re{};
    double m_im{};
  public:
    Complex(double re = 0, double im = 0)
      : m_re(re), m_im(im) {}
    double re() const { return m_re; }
    double im() const { return m_im; }

    double operator [] (size_t idx) {
      if (idx == 0) return m_re;
      if (idx == 1) return m_im;
      // ...
    }
};

Complex c {8, 9};
SHOW(c[0], c[1]);
```

```
c[0] = 8; c[1] = 9
```

Beispiel Ausgabeoperator

- Compiler erlaubt für `std::cout << c;` verschiedene Interpretationen
  - Methodenaufruf `std::ostream::operator<<(const Complex&);`
    - für primitive Datentypen als Methoden von `std::ostream`
  - Funktionsoperator `std::ostream& operator << (std::ostream&, const Complex&);`
    - für eigene Datentypen

```
class Complex {
    // Barton-Nackmann trick: friend FUNCTION inside class
    friend std::ostream& operator << (std::ostream& os, const Complex &c) {
      return os << '(' << c.m_re << '+' << c.m_im << "i)";
    }
  // ...
};
std::cout << Complex{1, 2} << ", "
          << Complex{1, 0} << ", "
          << Complex{0, 2} << std::endl;
// std::println("{}", c); XXX: not available
```

```
(1+2i), (1+0i), (0+2i)
```

Beispiel Prefix und Postfix Increment Operator

```
class Complex {
  public:
  // ...
    Complex& operator ++ () {
      std::println("prefix increment");
      m_re++;
      return *this;
    }

    Complex operator ++ (int) { // dummy int
      std::println("postfix increment");
      Complex temp {m_re, m_im};
      m_re++;
      return temp;
    }
};
```

```
Complex c {};
SHOW2(c.re(), c.im());

++c;
SHOW2(c.re(), c.im());

c++;
SHOW2(c.re(), c.im());
```

```
c.re() = 0; c.im() = 0
prefix increment
c.re() = 1; c.im() = 0
postfix increment
c.re() = 2; c.im() = 0
```

Beispiel Funktionsoperator

- Objekt der Klasse kann wie normale Funktion verwendet werden

```
class Complex {
  private:
    double m_re{};
    double m_im{};
  public:
    Complex(double re = 0, double im = 0)
      : m_re(re), m_im(im) {}
    double re() const { return m_re; }
    double im() const { return m_im; }

    Complex operator () (double scale) const {
      return Complex {m_re * scale, m_im * scale};
    }
};

Complex c {3, 4};
Complex s = c(2);
SHOW(s.re(), s.im());
```

```
s.re() = 6; s.im() = 8
```

Beispiel Typumwandlungsoperator (*cast operator*)

- Definition für Typ `type` als Methode `operator type()` ohne Rückgabetyp

```
class Complex {
  private:
    double m_re{};
    double m_im{};
  public:
    Complex(double re = 0, double im = 0)
      : m_re(re), m_im(im) {}
    double re() const { return m_re; }
    double im() const { return m_im; }

    operator double() const {
      return m_re;
    }

    operator std::string() const {
      return std::format("({}, {})", m_re, m_im);
    }
};
```

- implizite Typumwandlung

```
Complex c {3,4};
double x = c;
std::string s = c;
SHOW(x);
SHOW(s);
```

```
x = 3
s = "(3, 4)"
```

- explizite Casts

```
Complex c {3,4};
auto x = static_cast<double>(c);
auto s = static_cast<std::string>(c);
SHOW(x);
SHOW(s);
```

```
x = 3
s = "(3, 4)"
```

Vergleichsoperatoren (i)

- vor C++20: Überladen/Implementieren einzelner Operatoren
  - z.B. »`operator == ()`« und »`operator < ()`«
  - Ableitung anderer z.B.
    - »`operator != ()`« automatisch oder
    - »`operator <= ()`« durch `std::rel_ops` in Header <utility>

```
class Complex {
  // ...

  bool operator == (const Complex &other) const {
    return
      m_re == other.m_re &&
      m_im == other.m_im;
  }

  bool operator < (const Complex &other) const {
    return
      m_re < other.m_re &&
      m_im < other.m_im;
  }
};
```

```
// ...
Complex a {}
Complex b {1, 2};
SHOW(a == b);
SHOW(a < b);

// derived operators
SHOW(a != b);

#include <utility>
using namespace std::rel_ops;
SHOW(a <= b);
```

```
a == b = false
a < b = true
a != b = true
a <= b = true
```

Vergleichsoperatoren (ii)

- seit C++ 20: Spaceship-Vergleichsoperator
  - alle Vergleichsoperatoren automatisch abgeleitet
  - verschiedene Arten von Vergleichbarkeit
    - `std::strong_ordering`: alle Werte vergleichbar, Gleichheit $\Rightarrow$ Ersetzbarkeit ($\forall_{x,y} x = y \Rightarrow f(x) = f(y)$)
    - `std::weak_ordering`: alle Werte vergleichbar, keine Ersetzbarkeit garantiert
    - `std::partial_ordering`: nicht alle Werte vergleichbar, keine Ersetzbarkeit garantiert
  - `default`-Implementierung für `operator ==` möglich

```cpp
#include <compare>

class Complex {
  // ...
  using ord = std::partial_ordering;
  ord operator <=> (const Complex &o) const {
    if (m_re == o.m_re && m_im == o.m_im)
      return ord::equivalent;
    if (m_re < o.m_re && m_im < o.m_im)
      return ord::less;
    if (m_re > o.m_re && m_im > o.m_im)
      return ord::greater;
    return ord::unordered;
  }
  // NOTE: use `<=>` to implement `==` automatically
  bool operator == (const Complex&) const = default;
};
```

```cpp
Complex zero {};
Complex a {1, 2};
Complex b {1, 3};
SHOW(zero < a);
SHOW(a < b);
SHOW(b < a);
SHOW(a > b);
SHOW(a == b);
SHOW(a == a);
SHOW((a <=> b) == (b <=> a));
```

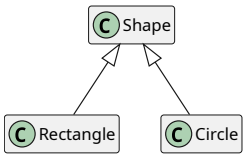
```
zero < a = true
a < b = false
b < a = false
a == b = false
a == a = true
(a <=> b) == (b <=> a) = true
```

11.4 C++ Vererbung

Vererbung

Vererbung ermöglicht

- Wiederverwendung und Erweiterung bestehender Implementierung
- Verallgemeinerung und Spezialisierung von Verhalten
- Vielgestaltigkeit (*polymorphism*)
  - gleicher Code mit verschiedenen Implementierungen
  - mit Hilfe von dynamische Bindung
  - erfordert Referenz oder Zeiger




```
Rectangle
Circle
```

```
struct Shape {
  // NOTE: pure virtual method[1]
  virtual void draw() = 0;
};

struct Rectangle : public Shape {
  void draw() override { PL("Rectangle"); }
};

struct Circle : public Shape {
  void draw() override { PL("Circle"); }
};

void draw(Shape &s) { // pass by reference
  s.draw(); // dynamic dispatch
}

int main() {
  Rectangle r {}; draw(r);
  Circle c {}; draw(c);
}
```

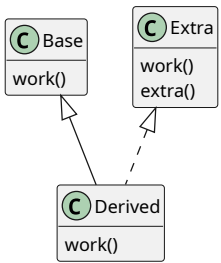
[1] `draw() = nullptr` wäre logischer ist aber syntaktisch nicht vorgesehen

Deklaration von Basisklassen

- C++ ermöglicht mehrfache Vererbung und Angabe von Sichtbarkeiten
  - z.B. nur Erben von Funktionalität aber keine gültige Spezialisierung
  - Unterscheidung von geerbten Methoden beim Aufruf durch Angabe der Basisklasse, z.B. `Base::work()`
  - keine allgemeine Referenz zur Basisklasse (Java `super`, C# `base`)

```cpp
struct Base {
  virtual void work() { PL("Base::work"); }
};
struct Extra {
  void work() { PL("Extra::work"); }
  void extra() { PL("Extra::extra"); }
};
struct Derived
  : public Base, // Derived is a Base
    private Extra { // Derived only inherits
  void work() override {
    Extra::work(); Base::work();
    extra();
  }
};
int main() {
  Derived d;
  d.work();
  // d.extra(); // XXX: private
}
```

```
Extra::work
Base::work
Extra::extra
```



Deklaration von Basisklassen

Liskovsches Ersetzbarkeitsprinzip[1] (*Liskov substitution principle*)

- abgeleitete Typen müssen allen Anforderungen des Basistyps entsprechen
- also an jeder Stelle verwendet werden können, wo die Basisklasse verwendet werden könnte

[1] B. Liskov, J. Wing *"Family Values: A Behavioral Notion of Subtyping"*. CMUCS93187, 1993

```
struct PocketKnife : private Knife, private File,
                     private Saw, private Pick {
    void work() {
      Knife::cut();
      File::cut();
      slice(); clean();
    }
};
PocketKnife pn {};
pn.work();
// pn.cut();      // ERROR: private method
// Knife &k = pn; // ERROR: private base class
```

```
cutting, smoothing, slicing, cleaning,
```

```
struct Knife {
  void cut() { std::print("cutting, "); }
};
struct File {
  void cut() { std::print("smoothing, "); }
};
struct Saw {
  void slice() { std::print("slicing, "); }
};
struct Pick {
  void clean() { std::print("cleaning, "); }
};
```

Konstruktor und Destruktor in Klassenhierarchie

- vor eigenem Konstruktor kommen Konstruktoren aller Basisklassen
- nach eigenem Destruktor kommen Destruktoren aller Basisklassen
- jeder Konstruktor initialisiert eigene Datenkomponenten
- jeder Destruktor deinitialisiert eigene Datenkomponenten

```
struct Value {
  std::string m_s; // 1.1 & 2.1
  Value(std::string s) : m_s(s) { // 1.1 & 2.1
    std::println("Value({})", s);
  }
  ~Value() { std::println("~Value({})", m_s); }
};

struct Base {
  Value m_base_value { "Base" }; // 1.1
  Base() { std::println("Base()"); } // 1.2
  ~Base() { std::println("~Base()"); }
};

struct Derived : public Base {
  Value m_derived_value { "Derived" }; // 2.1
  Derived() { std::println("Derived()"); } // 2.2
  ~Derived() { std::println("~Derived()"); }
};
```

```
Derived d {}; std::println("----");
```

```
Value(Base)
Base()
Value(Derived)
Derived()
----
~Derived()
~Value(Derived)
~Base()
~Value(Base)
```

1. Base
   1.1 Daten initialisieren
   1.2 Konstruktor aufrufen
2. Derived
   2.1 Daten initialisieren
   2.2 Konstruktor aufrufen

Konstruktoraufruf delegieren

- Konstruktor kann zu Basisklasse oder eigenem anderem Konstruktor delegiert werden

```
struct Base {
  Base(int x) {
    PL("Base({})", x);
  };
};
struct Derived : public Base {

  Derived(int x) : Base(x) {
    PL("Derived({})", x);
  }

  Derived() : Derived(0) {
    PL("Derived()");
  }

};
int main() {
  Derived d;
}
```

```
Base(0)
Derived(0)
Derived()
```

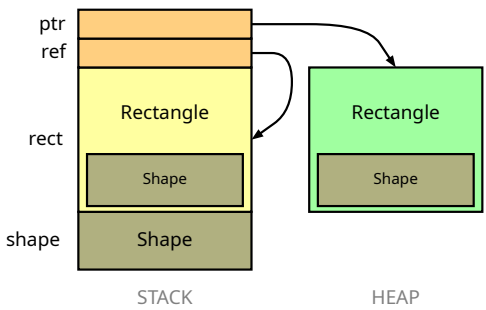
Polymorphismus & Dynamische Bindung (i)

- statischer Typ ist Zeiger oder Referenz der Basisklasse
- dynamischer Typ ist zur Laufzeit die korrekte Klasse
- Referenz oder Zeiger notwendig da Speicherbedarf für abgeleitete Klasse nicht immer bekannt

```
class Shape {
  /* ... */
};

class Rectangle : public Shape {
    /* ... */
};

int main() {
  Shape shape {};
  Rectangle rect {};
  Shape &ref = rect;
  Shape *ptr = new Rectangle{};
  delete ptr;
}
```



- Abgeleitete Klasse enthält ebenfalls alle Datenkomponenten der Basisklasse(n)

Polymorphismus & Dynamische Bindung (ii)

- dynamisch gebundene Methode → zur Laufzeit Implementierung auswählen
- Methode wird dazu als `virtual` gekennzeichnet und kann überschrieben werden
- Methode in abgeleiteten Klassen automatisch `virtual`
  - mit Schlüsselwort `override` sicherstellen (empfohlen aber optional)

```
struct Base {
  virtual void virtual_function_1() {}
  virtual void virtual_function_2() {}
  virtual void virtual_function_3() {}
  virtual void virtual_function_4() {}
  virtual void virtual_function_5() {}
};
struct Derived : public Base {
  virtual void virtual_function_1() {}
          void virtual_function_2() {}          // NOTE: also virtual
          void virtual_function_3() override {} // NOTE: better
          void vitrual_fnuctoin_4() override {} // ERROR: typo caught by compiler
          void vitrual_fnuctoin_5() {}          // NOTE: no error found
};
```

```
error: ‘Derived::vitrual_funtcoin_4()’ marked ‘override’, but does not override
   26 |           void vitrual_fnutcoin_4() override {}
      |                ^~~~~~~~~~~~~~~~~~
```

Polymorphismus & Dynamische Bindung (iii)

- Compiler erzeugt Tabelle mit dynamisch gebundenen Methoden (*virtual table* – `vtab`)
  - Objekt enthält Zeiger auf korrekte Tabelle der Klasse
  - Aufruf indirekt über Tabelle → daher (etwas) langsamer
- Abstrakte Methode (ohne Implementierung) daher Null-Zeiger (`=0`)

```
struct Shape {
  virtual void draw() = 0; // abstract method
  virtual void print() { /* implementation for all Shapes */ }
  // ...
};

struct Rectangle {
  void draw() override { /* implementation for Rectangle */ }
};

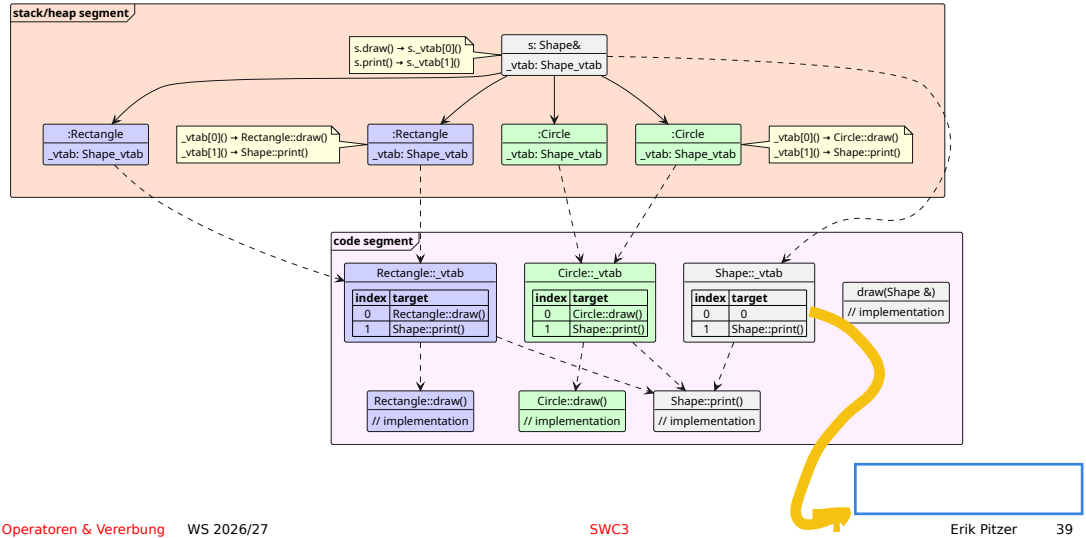
struct Circle {
  void draw() override { /* implementation for Circle */ }
};
```

```
void draw(Shape &s) {
  s.draw(); // --> ???
  s.print(); // --> ???
}
```

implementiert als →

```
void draw(Shape &s) {
  s._vtab[0]();
  s._vtab[1]();
}
```

Polymorphismus & Dynamische Bindung (iv)



Dynamisch gebundener Desktruktor

- bei dynamischem Speicher, Polymorphismus des Desktrukors berücksichtigen

- Typ `Base*` ergibt (statisch) nur Aufruf von `~Base()`

```
struct Base {
  Base() { std::print("Base(), "); }
  ~Base() { std::print("~Base(), "); }
};
struct Derived : public Base {
  Derived() { std::print("Derived(), "); }
  ~Derived() { std::print("~Derived(), "); }
};
Derived *d = new Derived(); delete d;
std::println();
Base *b = new Derived(); delete b;
```

```
Base(), Derived(), ~Derived(), ~Base(),
Base(), Derived(), ~Base(),
```

- fehlender Aufruf von `~Derived()` vor Aufruf von `~Base()`

- mit `virtual` Desktruktor korrekte Freigabe, auch von `Derived`

```
struct Base {
  Base() { std::print("Base(), "); }
  virtual ~Base() { std::print("~Base(), "); }
};
struct Derived : public Base {
  Derived() { std::print("Derived(), "); }
  ~Derived() { std::print("~Derived(), "); }
};
Derived *d = new Derived(); delete d;
std::println();
Base *b = new Derived(); delete b;
```

```
Base(), Derived(), ~Derived(), ~Base(),
Base(), Derived(), ~Derived(), ~Base(),
```

- Destruktor `~Derived()` ist automatisch `virtual`, auch ohne explizite Angabe

Konvertierung entlang der Klassenhierarchie (*upcast* und *downcast*)

```cpp
struct Shape { virtual ~Shape() {} };
struct Rectangle : public Shape {};
struct Circle : public Shape {};

int main() {
  Rectangle r {};
  Circle c {};

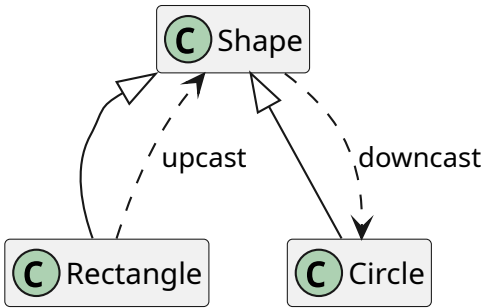
  // upcast
  Shape *s = &c;

  // downcasts
  Rectangle *r2 = dynamic_cast<Rectangle*>(s);
  if (r2 != nullptr)
    std::println("s was Rectangle");

  Circle *c2 = dynamic_cast<Circle*>(s);
  if (c2 != nullptr)
    std::println("s was Circle");
}
```

```
s was Circle
```

- upcast zu Basisklasse → immer möglich
- downcast zu abgeleiteter Klasse → *kann fehlschlagen*
  - mit Hilfe von `dynamic_cast<>`



Mehrfachvererbung & das Diamant-Problem

```cpp
struct Base {
  int value {123};
  virtual void print() const {
    std::print("Base, ");
  }
};

struct A : public Base {
    void print() const override {
      Base::print();
      std::println("A");
    }
};

struct B : public Base {
    void print() const override {
      Base::print();
      std::println("B");
    }
};

struct Derived : public A,
                 public B {
};
```

```cpp
int main() {
  Derived d;
  // d.print(); // XXX: error: ambiguous
  A(d).print();
  B(d).print();
}
```

```
Base, A
Base, B
```

- Klasse `Derived` erbt von `Base` zweimal
  - einmal über `A`
  - einmal über `B`
- enthält somit zwei Kopien von `value`
- Abhilfe durch virtuelle Vererbung
  - z.B. `struct A : virtual public Base`
- Mehrfachvererbung eher vermeiden

